

TCS-504 System Design Assignment 1

Movie Ticket Booking System

Semester	5
Subject	System Design
Subject Code	TCS-504
Submission Date	07 September 2026
Student Name	Sahil Singh Gandhi
Section	G1
Roll No.	2419505

A formal system design report for a small menu-driven C++ console program for one cinema.

1. Scope and Problem Statement

The assignment is to design and implement a small movie ticket booking system for a single cinema as a menu-driven C++ console program. A customer can view movies, choose a show, inspect seat availability, book seats, make payment, receive a printed ticket, and cancel a confirmed booking.

The scope is limited to the required booking workflow. The design does not add unsupported features such as online accounts, refunds, loyalty points, multiple cinemas, databases, web interfaces, or notification systems.

2. Requirement Analysis

Functional Requirements

ID	Functional Requirement
FR1	The system shall display all movies currently playing in the cinema, including title, language, and duration.
FR2	For a selected movie, the system shall display all available shows with screen name and start time.
FR3	For a selected show, the system shall display every show seat as AVAILABLE or BOOKED before booking.
FR4	The customer may select one or more seats for a show. If any selected seat is already BOOKED, the entire booking shall be rejected and no seat state shall change. Booking is confirmed only after successful payment.
FR5	The system shall calculate the total amount using exact seat prices: SILVER Rs.150, GOLD Rs.250, and PLATINUM Rs.400.
FR6	The system shall allow exactly one payment method per booking: UPI, Card, or Cash. If payment fails, selected seats shall be released and booking status shall become FAILED.
FR7	After successful payment, the system shall print a ticket containing booking ID, movie, screen, time, seats, total amount, and status.
FR8	The system shall allow cancellation of a confirmed booking and shall make the cancelled booking seats AVAILABLE again.

Non Functional Requirements

ID	Quality	Non Functional Requirement
NFR1	Modularity	The C++ implementation shall use one class per file and no header files, following the assignment structure.
NFR2	Extensibility	New payment types should be addable by creating a new Payment subclass without changing existing payment classes.
NFR3	Input validation	Invalid menu choices, invalid show choices, and invalid seat numbers shall be handled with clear messages and no crash.
NFR4	Maintainability	Names shall be meaningful, prices shall use constants, and ticket printing shall remain separate from booking data.

3. Noun Verb Analysis

Noun Found	Keep as Class	Reason
Movie	Yes	Has its own identity and stores title, language, and duration.
Seat	Yes	Represents a physical seat with seat number and category.
Screen	Yes	Represents an auditorium and owns physical seats.
Cinema	Yes	Represents the single theatre and owns screens.
Show	Yes	Combines a movie, screen, and start time.
ShowSeat	Yes	Stores AVAILABLE or BOOKED status for one physical seat in one show.
Customer	Yes	Stores customer name and phone number.
Booking	Yes	Stores booking ID, customer, show, seats, amount, and status.
Payment	Yes	Acts as the abstract payment contract for UPI, Card, and Cash.
UPI/Card/Cash	Yes	Concrete payment methods with different pay implementations.
Price	No	It is a calculated value handled by PriceCalculator, not an independent class.
Ticket	No	The ticket is printed output; TicketPrinter handles the formatting.
Seat layout	No	It is a view of ShowSeat statuses, not a stored domain object.
BookingService	Yes	Coordinates validation, pricing, payment, booking, and cancellation.

Verb or Action	Handled By
list movies	Cinema data and main menu
list shows	Main menu using Show list
display seats	Show.displaySeatLayout()
book seats	BookingService.bookSeats() and ShowSeat.bookSeat()
calculate price	PriceCalculator.calculateTotal()
pay	Payment subclasses
print ticket	TicketPrinter.printTicket()
cancel booking	BookingService and Booking.cancelBooking()

4. Class Responsibilities

Class	What It Knows	What It Does	Must NOT Do
Movie	title, language, duration	Provides movie details	Manage shows, seats, or payment
Seat	seatNumber, seatType	Represents a physical seat	Store show-wise booking status
Screen	screenNumber, seats	Owns and exposes physical seats	Calculate payment or print tickets
Cinema	cinemaName, screens	Owns screens for one cinema	Handle booking workflow
Show	movie, screen, startTime, showSeats	Displays layout and finds show seats	Own movie or screen lifetime
ShowSeat	physical seat, status	Books or releases one show seat	Calculate total ticket amount
Customer	name, phone	Stores customer identity	Validate seats or make payment
Booking	bookingId, customer, show, seats, amount, status	Confirms, fails, or cancels booking	Print ticket or process payment
Payment	payment contract	Defines pay(amount)	Know booking internals
UpiPayment	success simulation flag	Pays using UPI behavior	Change seat status
CardPayment	success simulation flag	Pays using Card behavior	Change seat status
CashPayment	success simulation flag	Pays using Cash behavior	Change seat status
PriceCalculator	seat price constants	Calculates total amount	Book seats or print tickets
TicketPrinter	ticket formatting logic	Prints ticket details	Change booking status
BookingService	PriceCalculator, TicketPrinter, booking list	Orchestrates booking and cancellation	Store movie title or seat category itself

5. Relationship Table With Lifetime Test Justification

Pair	Relationship	Lifetime Test Justification
Cinema - Screen	Composition	If the Cinema object is removed, its screens are no longer part of this system model; Cinema owns its screen collection.
Screen - Seat	Composition	Seats belong to a physical screen in this model. Destroying Screen destroys its Seat objects.
Show - Movie	Aggregation	A Movie can exist independently and can be used by multiple shows; destroying one Show does not destroy the Movie.
Show - Screen	Association	A Show uses one Screen at a time; the Screen exists independently and can host other shows.
Show - ShowSeat	Composition	ShowSeat status exists only for one specific show. Destroying the Show destroys its ShowSeat objects.
Booking - Customer	Association	Booking stores customer details for a transaction, but Customer identity is not owned by the booking workflow as a whole.
Booking - ShowSeat	Aggregation	Booking references selected ShowSeat objects. Seats must continue to exist after booking cancellation.
Booking - Payment	Association	Payment is used during checkout to confirm or fail booking; Booking does not own every possible payment implementation.
Payment - UpiPayment	Inheritance	UpiPayment is a specialized Payment and must satisfy the same pay(amount) contract.
BookingService - Booking	Association	BookingService creates and manages booking workflow records but is a service, not a physical part-whole object.

6. UML Class Diagram

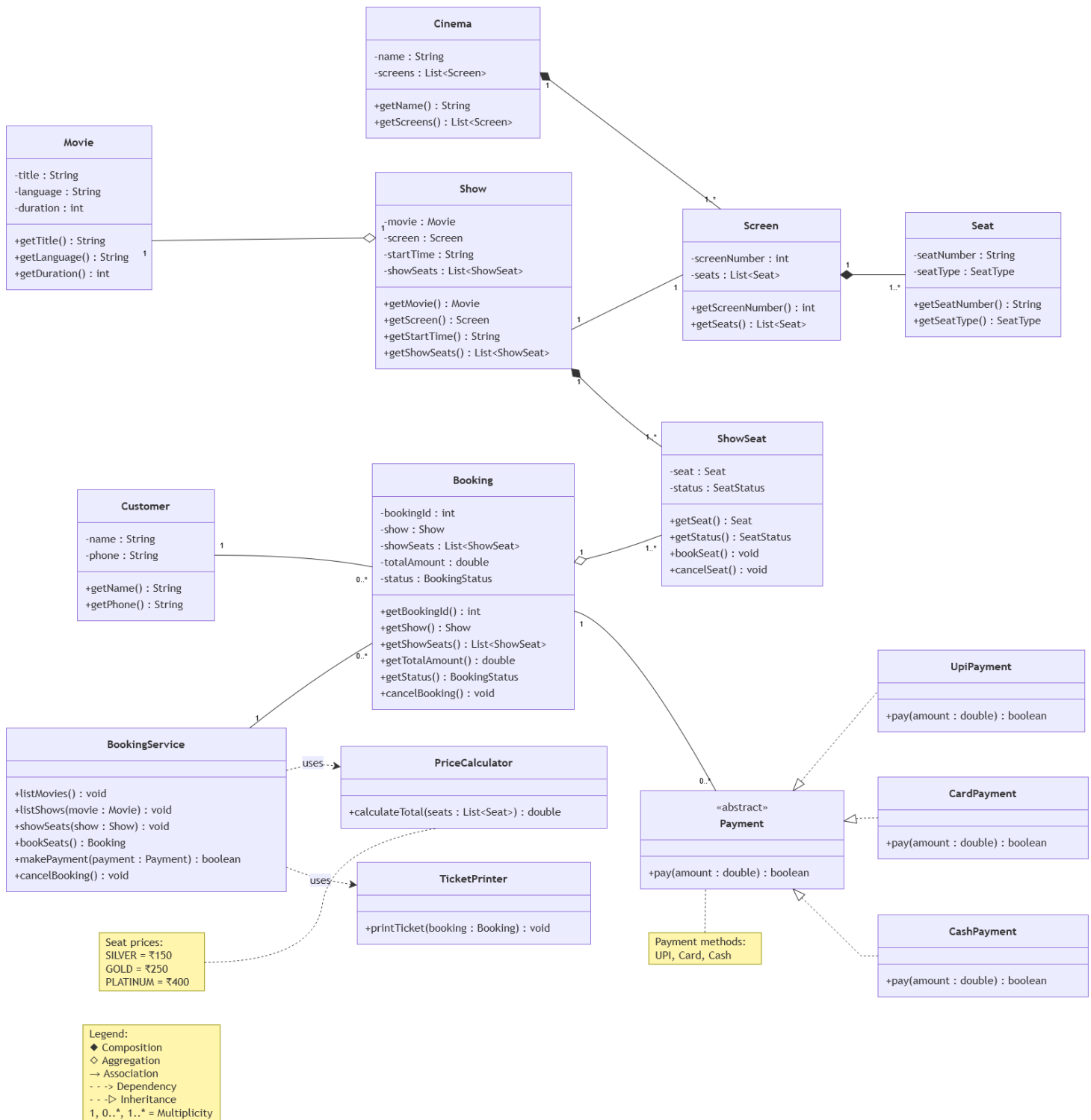


Figure 1. UML Class Diagram for Movie Ticket Booking System

7. Sequence Diagram

This sequence diagram shows the required use case: customer books 1 seat and pays by UPI. It includes the required lifelines, activation bars, synchronous calls, return messages, object creation messages for Payment and Booking, and alternative behavior for an already booked seat or failed payment.

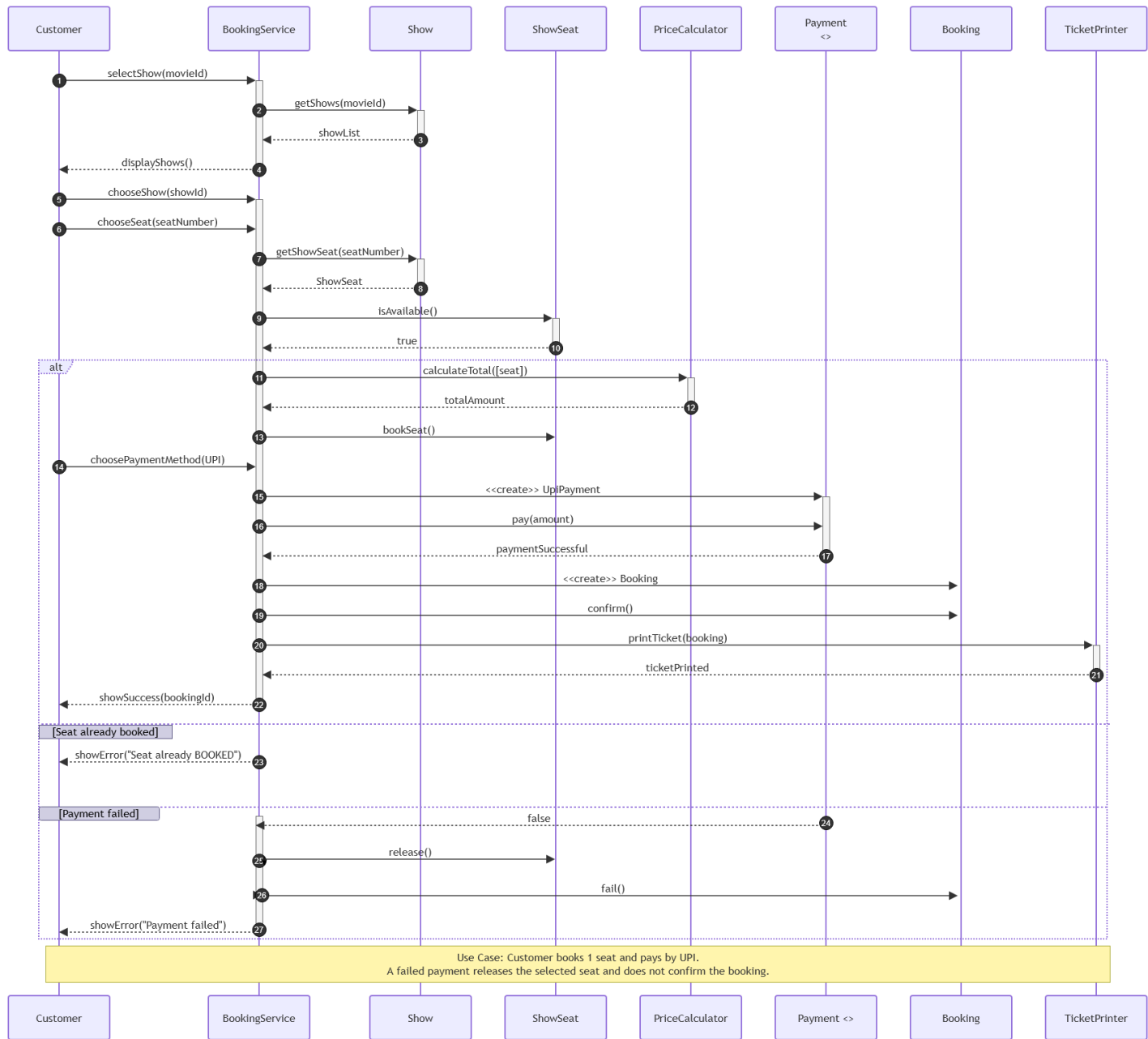


Figure 2. Sequence Diagram for Customer Books 1 Seat and Pays by UPI

8. OOP Concepts Used

Concept	Where Used
Encapsulation	Private data in Booking, ShowSeat, Movie, Seat, Screen, Cinema, and Customer; status changes through methods.
Abstraction	Payment is abstract with a pay(amount) contract.
Inheritance	UpiPayment, CardPayment, and CashPayment inherit from Payment.
Runtime polymorphism	BookingService receives a Payment reference and calls pay(total) without depending on the concrete payment method.
Compile-time polymorphism	Movie can use overloaded constructors or methods with different parameter lists.
Static members	Booking uses a shared next booking number to create unique booking IDs.
this keyword	Constructors use this to assign current object state clearly.
Composition	Cinema owns Screen, Screen owns Seat, and Show owns ShowSeat.
Aggregation	Show references existing Movie; Booking references selected ShowSeat.
Association	Customer interacts with BookingService; Booking uses Payment during checkout.

9. SOLID Principles

Principle	Application
Single Responsibility	TicketPrinter only formats tickets; Booking only stores and updates booking state; PriceCalculator only calculates price.
Open Closed	A new payment method can be added by creating another Payment subclass without changing UPI, Card, or Cash classes.
Liskov Substitution	Any Payment child can be passed through the Payment abstraction and used through pay(amount).
Interface Segregation	A refund method is not forced into Payment because the assignment only requires payment, failed payment release, and cancellation seat release.
Dependency Inversion	BookingService depends on the Payment abstraction instead of directly depending on a concrete UPI, Card, or Cash object.
Deliberate non choice	A refund system, database layer, web UI, and multiple-cinema search are not added because they are outside the assignment scope.

10. Clean Code Considerations

Rule	Implemented Evidence
Meaningful names	Examples include BookingService, PriceCalculator, selectedSeats, and totalAmount.
No numbered variables	Avoid list1, temp1, and similar names that hide purpose.
One function, one job	Separate methods handle input, movie choice, show choice, payment, cancellation, and printing.
Short functions	Core methods should remain focused; the booking workflow is divided into helper steps.
Limited indentation	Validation uses early rejection for invalid input and unavailable seats.
Simple parameters	Classes carry related state; large primitive parameter lists are avoided.
No unwanted side effects	displaySeatLayout() prints availability only and does not book seats.
No magic prices	SILVER_PRICE, GOLD_PRICE, and PLATINUM_PRICE constants represent exact assignment prices.
DRY	Ticket formatting and price calculation are centralized.

11. Edge Cases and Testing

Case	Expected Result	Observed Result
Invalid menu choice	Show clear error and continue	Program prints an invalid choice message and returns to the menu.
Successful booking	Book available seats after successful UPI payment	Selected seats are booked, the total is calculated, and a confirmed ticket is printed.
Booked-seat rejection	Reject entire booking and do not change seat state	Second attempt for the same seat prints already BOOKED and rejects the booking.
Cancellation	Confirmed booking can be cancelled and seats become available	Cancelled booking seats return to AVAILABLE.
Failed payment	Booking is not confirmed and selected seats are released	Payment failure creates a failed booking state and releases selected seats.
Invalid seat number	Show clear error and do not crash	Invalid seat input prints an error message.

Selected Demo Output

```
===== MOVIE TICKET BOOKING =====
1. Movies
2. Book
3. Cancel
4. My tickets
0. Exit
Choice: Invalid menu choice.

===== MOVIE TICKET BOOKING =====
1. Movies
2. Book
3. Cancel
4. My tickets
0. Exit
Choice:
Currently Playing Movies
[1] 3 Idiots      Hindi      170 min
[2] Interstellar  English    169 min
Select movie:
Shows for 3 Idiots
[1] Screen-1     06:00 PM
[2] Screen-2     09:00 PM
Select show:
Screen-1  06:00 PM   3 Idiots

[ ] = AVAILABLE, [X] = BOOKED
SILVER   A1[ ] A2[ ] A3[ ] A4[ ]
GOLD     B1[ ] B2[ ] B3[ ] B4[ ]
PLATINUM C1[ ] C2[ ]
Customer name: Phone: Seats (example A1,B2):
Pay by:
1. UPI
2. Card
3. Cash
Choice: Simulate payment success? 1.Yes 2.No:
Price Breakup
A1 SILVER    Rs.150
B2 GOLD      Rs.250
TOTAL        Rs.400
[UPI] Rs.400 paid successfully

===== TICKET =====
Booking ID : BK1001
Customer   : Rahul Sharma
Movie      : 3 Idiots
Screen     : Screen-1
Time       : 06:00 PM
Seats      : A1, B2
Amount     : Rs.400
Status     : CONFIRMED
=====
```

12. Conclusion

The design satisfies the required scope for the Movie Ticket Booking System. The responsibilities are separated across entity, behavior, and service classes; relationships are justified using the lifetime test; and the payment hierarchy supports abstraction, inheritance, and runtime polymorphism. The required edge cases are represented in the design and are suitable for demonstration in the modular C++ implementation.